

ENGENHARIA DE SOFTWARE II · 2025

RANK-IA

Sua organização de maneira eficiente

Integrantes

Fernando Hiroshi Murusaki · RA 241025851

Igor dos Reis Gomes · RA 241025265

Matheus Santos Magro · RA 231025335

Murilo Tomaz Gonzaga · RA 241024684

Grupo

Grupo 8

UNESP Bauru · BCC

O QUE É O RANK-IA?

Formação inteligente de equipes corporativas

Aplicação web corporativa que utiliza **Reinforcement Learning** para montar equipes otimizadas com base em habilidades, competências e gaps dos colaboradores.

- Geração automática de equipes com **Coeficiente de Aproveitamento**
- Ajustes manuais que retroalimentam o modelo de IA
- Sugestão de treinamentos baseados em gaps identificados
- Relatórios de desempenho individual e coletivo

PERFIL

Gestor / RH

Acesso total: cadastro, config. da IA, validação e relatórios

PERFIL

Colaborador

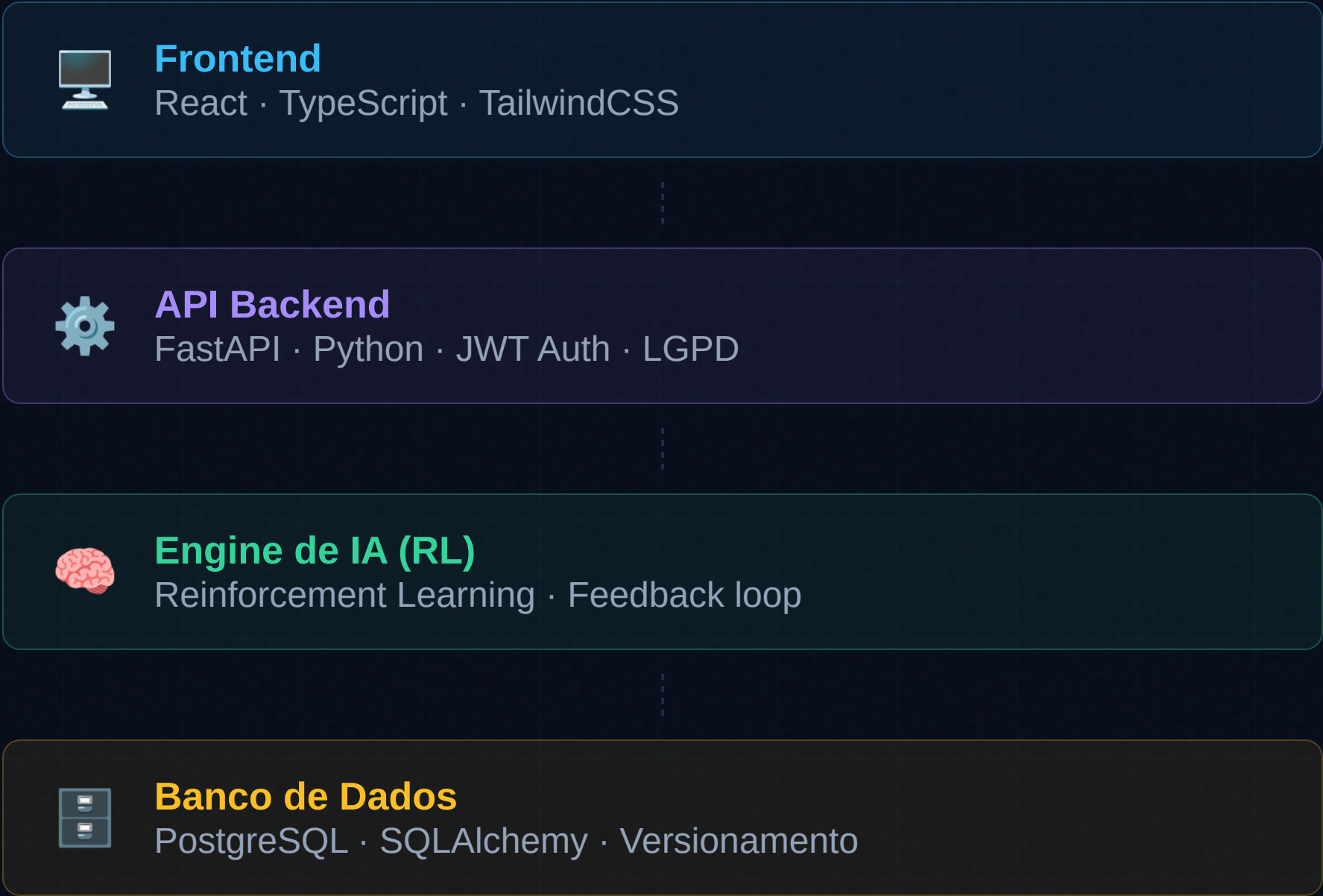
Consulta de competências, treinamentos e equipes

METODOLOGIA

Scrumban

Scrum + Kanban · Sprints de 2 semanas

Arquitetura do RANK-IA



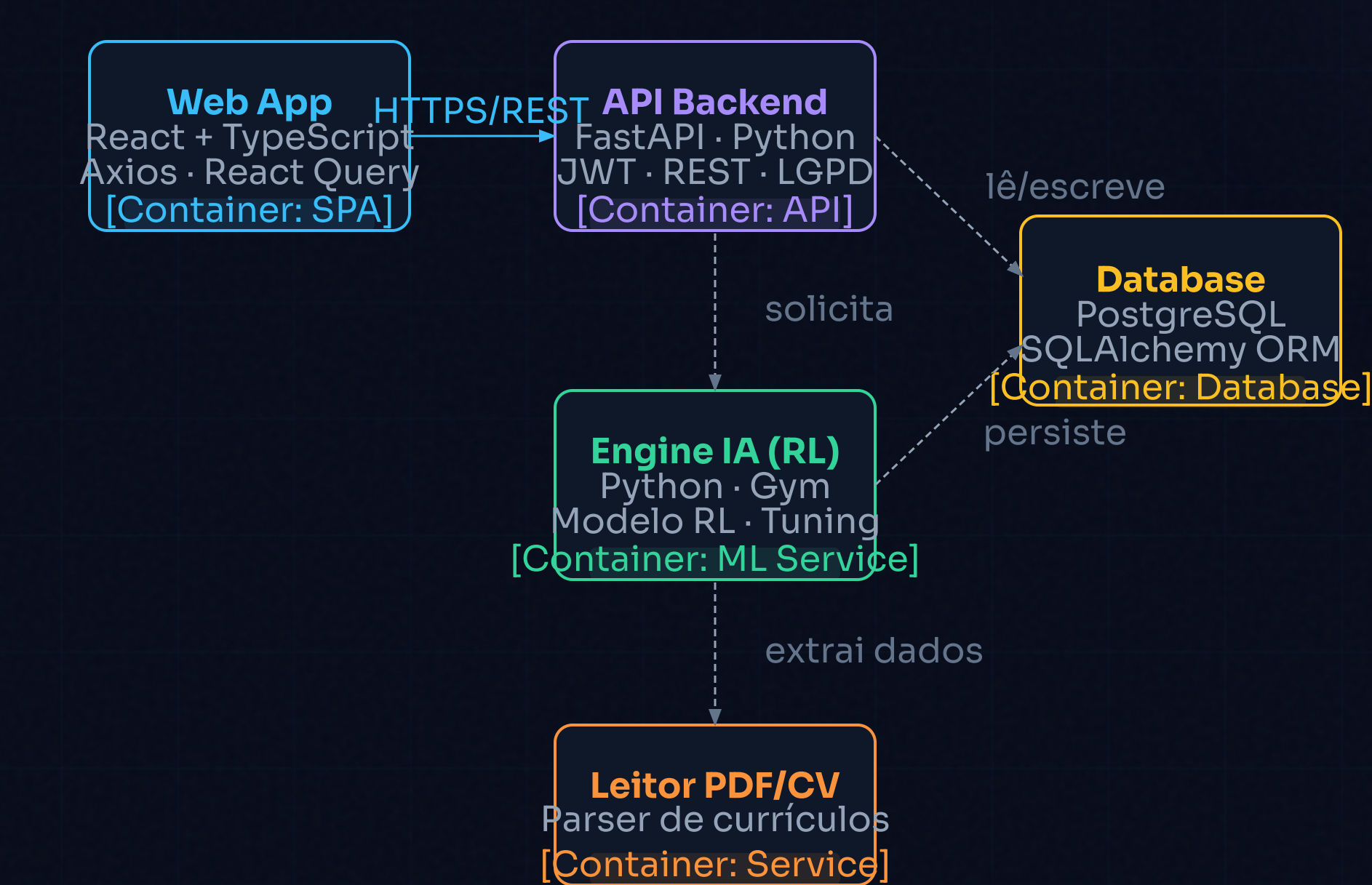
MÓDULOS PRINCIPAIS



Contexto do Sistema



Diagrama de Containers



LEGENDA DE CONTAINERS

- Single Page Application (React)
- REST API (FastAPI)
- ML Service (Reinforcement Learning)
- Database (PostgreSQL)
- Serviço de leitura de CV/PDF

Comunicação

SPA ↔ API via HTTPS/REST
API ↔ IA via chamadas internas
API ↔ DB via SQLAlchemy ORM
Dados criptografados em trânsito

Diagrama de Classes Geral

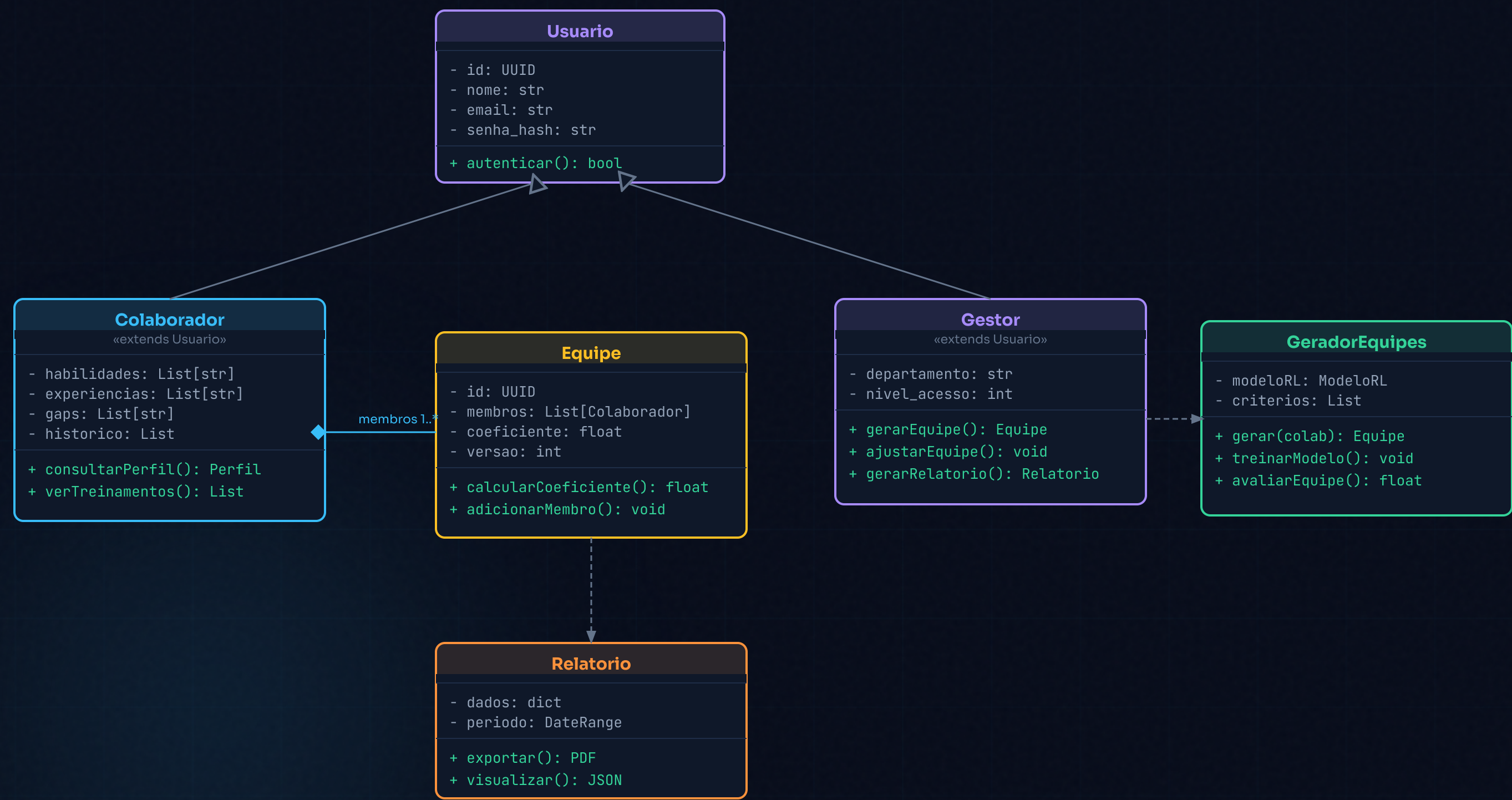
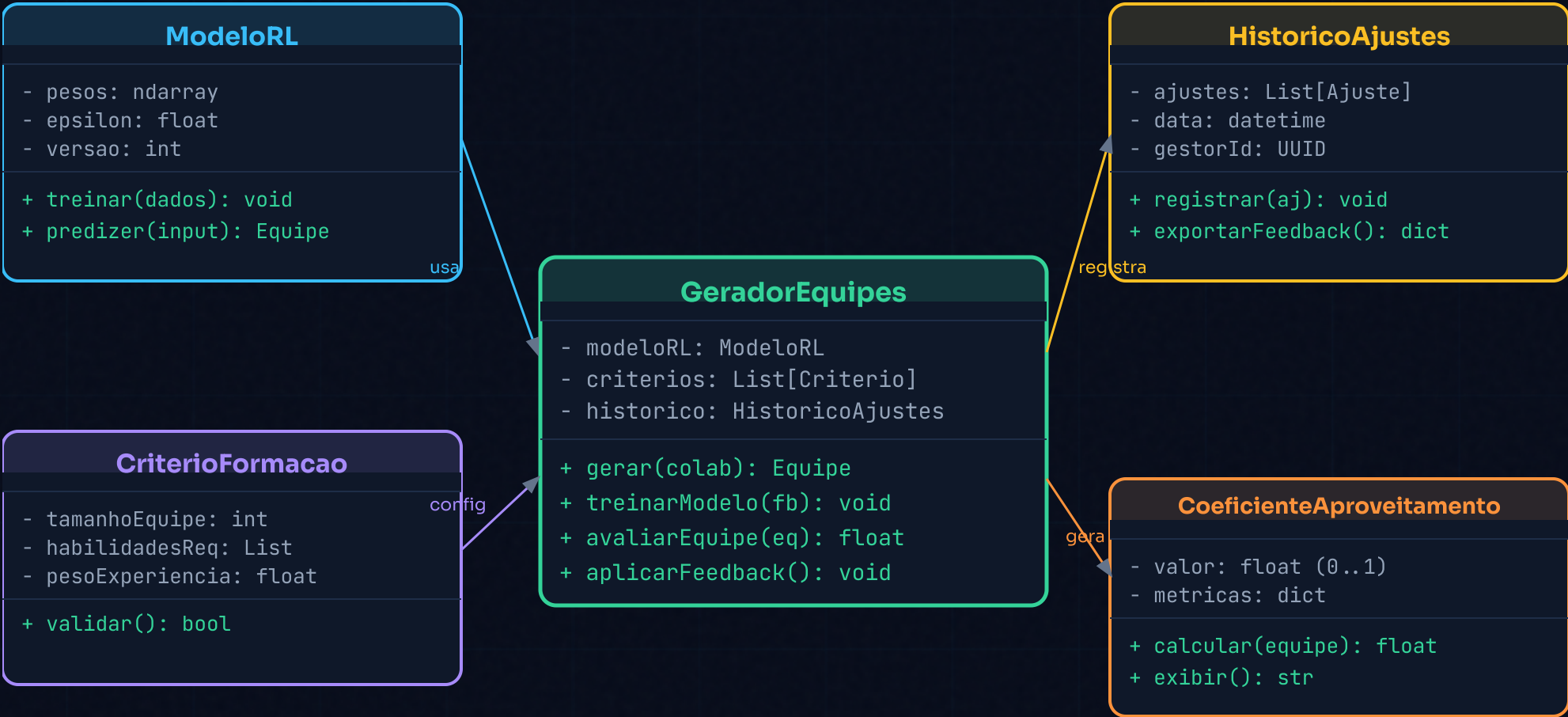


Diagrama Específico: GeradorEquipes



Por que detalhar GeradorEquipes?

É o coração do sistema: responsável por receber os colaboradores, aplicar o modelo de RL, gerar equipes e calcular o Coeficiente de Aproveitamento.

ModeloRL

Encapsula os pesos e hiperparâmetros do algoritmo de RL

CriterioFormacao

Define os parâmetros e requisitos para formação da equipe

HistoricoAjustes

Registra mudanças manuais para retroalimentar o modelo

CoeficienteAproveitamento

Calcula e expõe a métrica de qualidade da equipe gerada

Código: GeradorEquipes

```
from dataclasses import dataclass
from typing import List
from uuid import uuid4
import numpy as np

@dataclass
class CriterioFormacao:
    tamanho_equipe: int
    habilidades_req: List[str]
    peso_experiencia: float = 0.5

    def validar(self) → bool:
        return self.tamanho_equipe > 0

@dataclass
class CoeficienteAproveitamento:
    valor: float = 0.0

    def calcular(self, membros: List) → float:
        # Média ponderada: habilidades vs gaps
        scores = [
            len(m.habilidades) / (len(m.gaps) + 1)
            for m in membros
        ]
        self.valor = round(float(np.mean(scores)), 3)
        return self.valor
```

```
class ModeloRL:
    def __init__(self):
        self.pesos = np.random.rand(10)
        self.epsilon = 0.1
        self.versao = 1

    def predizer(self, colaboradores, criterio):
        # Seleciona os n melhores colaboradores
        pontuados = sorted(
            colaboradores,
            key=lambda c: len(c.habilidades),
            reverse=True
        )
        return pontuados[:criterio.tamanho_equipe]

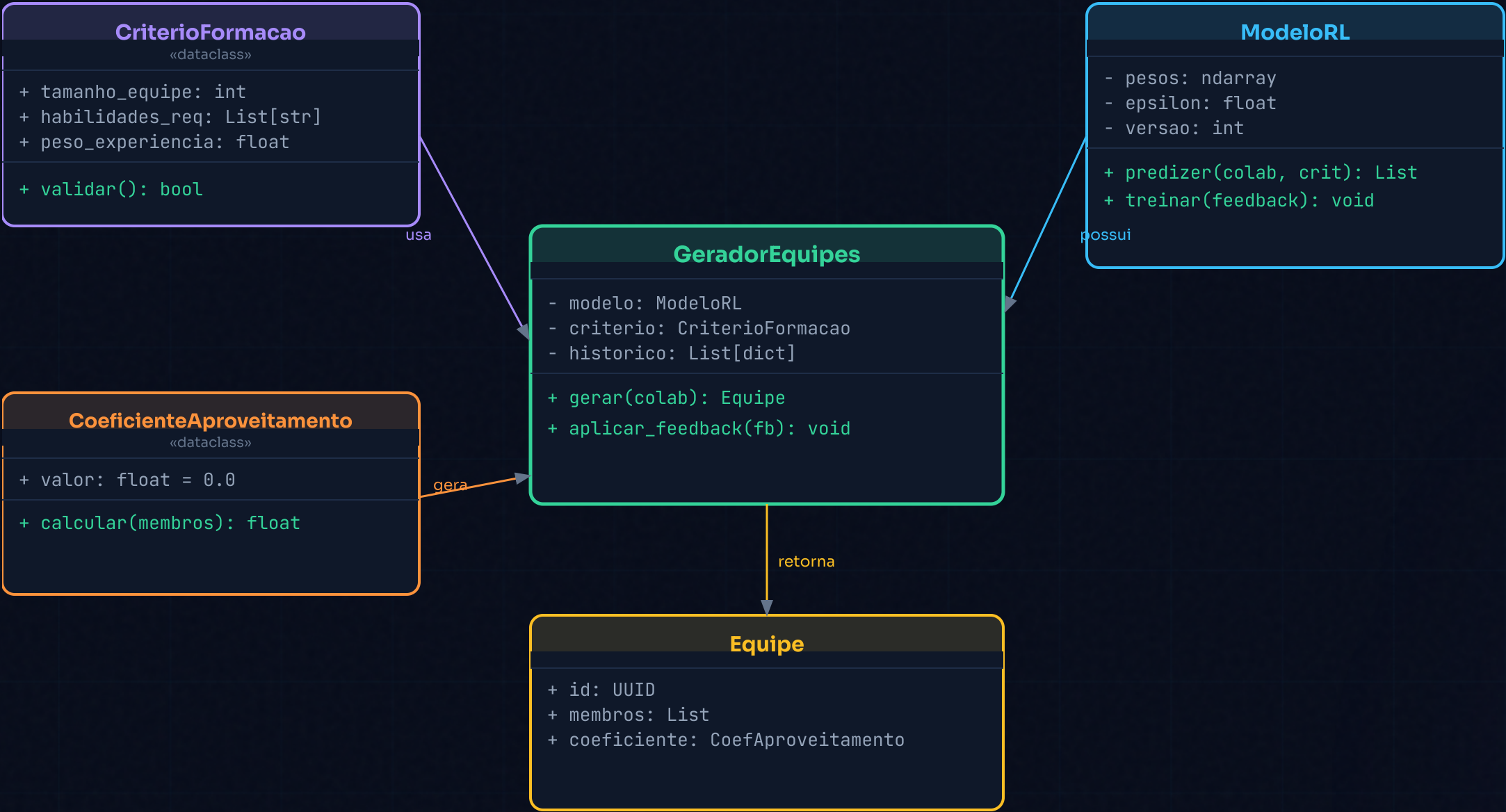
    def treinar(self, feedback: dict):
        vals = np.array(list(feedback.values()))[:10]
        self.pesos += 0.01 * vals
        self.versao += 1

class GeradorEquipes:
    def __init__(self, criterio: CriterioFormacao):
        self.modelo = ModeloRL()
        self.criterio = criterio
        self.historico = []

    def gerar(self, colaboradores) → Equipe:
        membros = self.modelo.predizer(
            colaboradores, self.criterio)
        ca = CoeficienteAproveitamento()
        ca.calcular(membros)
        return Equipe(uuid4(), membros, ca)

    def aplicar_feedback(self, fb: dict):
```


Diagrama de Classes do Código



Correspondência código ↔ diagrama

- @dataclass → classes com campos tipados
- def gerar() → método central com retorno Equipe
- Composição: CriterioFormacao injetado no construtor
- ModeloRL instanciado internamente (acoplamento)

Observação para o próximo passo

Note que GeradorEquipes instancia ModeloRL diretamente — isso viola o princípio de Inversão de Dependência do SOLID. Veremos como corrigir isso!

O que é SOLID?

S

Single Responsibility

Uma classe deve ter **apenas uma razão para mudar**. Cada classe é responsável por uma única parte da funcionalidade.

O

Open / Closed

Aberta para extensão, fechada para modificação. Adicione novos comportamentos sem alterar código já existente.

L

Liskov Substitution

Subclasses devem poder **substituir suas superclasses** sem alterar o comportamento esperado do programa.

I

Interface Segregation

Interfaces devem ser **pequenas e específicas**. Clientes não devem depender de métodos que não utilizam.

D

Dependency Inversion

Módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de **abstrações (interfaces)**.

Importância do SOLID no RANK-IA

S

Single Responsibility

GeradorEquipes só gera equipes. ServicoRelatorio só gera relatórios. ServicoNotificacao só notifica. Cada serviço evolui independentemente.

O

Open / Closed

Novos algoritmos de avaliação podem ser adicionados implementando IValidadorEquipe sem modificar o core do GeradorEquipes.

L

Liskov Substitution

AvalidadorIA e AvalidadorRegras são intercambiáveis — o sistema funciona corretamente com qualquer implementação de IValidadorEquipe.

I

Interface Segregation

Interfaces separadas: IRepositorioEquipe, IValidadorEquipe, INotificador. Cada componente depende só do que usa.

D

Dependency Inversion

GeradorEquipes recebe IValidadorEquipe via injeção — não conhece a implementação concreta. Fácil de testar com mocks.

BENEFÍCIOS NO CONTEXTO DO RANK-IA

- Trocar o modelo de RL por outro algoritmo sem quebrar o sistema
- Testar cada serviço isoladamente com mocks e stubs
- Equipe pode trabalhar em paralelo em serviços distintos sem conflitos
- Adicionar novos critérios de formação sem risco de regressão
- Conformidade com boas práticas → código auditável (LGPD)

Resumo da transformação

God Class

→

Serviços SOLID

PROBLEMA IDENTIFICADO — VIOLAÇÕES DE SOLID

Código **Antes** do SOLID

```
# X God Class — viola S, O e D
class GerenciadorDeEquipes:
    def __init__(self, db_conn):
        self.db = db_conn
        # X instancia diretamente — viola D
        self.modelo_ia = ModeloRL()
        self.smtp = SMTPClient("smtp.corp.com")

# X gera equipe E persiste E notifica (viola S)
def gerar_equipe(self, colaboradores, criterios):
    membros = self.modelo_ia.predizer(colaboradores)
    equipe = Equipe(membros)
    # X BD dentro do gerador
    self.db.execute("INSERT INTO equipes...", equipe)
    # X e-mail dentro do gerador
    self.smtp.send(destinatarios, equipe)
    return equipe

# X relatório misturado no mesmo lugar
def gerar_relatorio(self, equipe):
    dados = self.db.query("SELECT * FROM ...")
    return PDFExporter().exportar(dados)

# X treinamento da IA aqui também
def treinar_modelo(self, feedback):
    self.modelo_ia.atualizar(feedback)

# X sugestão de treinamentos no gerador?!
def sugerir_treinamentos(self, colaborador):
    return self.db.query(
        "SELECT * FROM treinamentos"
        " WHERE area IN ?", colaborador.gaps
    )
```

Problemas encontrados

- X S — Múltiplas responsabilidades**
Uma única classe gera equipes, persiste no BD, envia e-mail, gera relatórios e sugere treinamentos.
- X O — Difícil de estender**
Para trocar o algoritmo de IA, é necessário modificar diretamente a classe, arriscando quebrar outras funções.
- X D — Dependência concreta**
`ModeloRL()` e `SMTPClient()` instanciados internamente — impossível de mockar nos testes.
- X Consequências práticas**
Impossível testar isoladamente. Qualquer mudança pode quebrar partes não relacionadas. Código frágil.

Código Depois do SOLID

```
from abc import ABC, abstractmethod

# ✔ D — Abstrações (interfaces)
class IAvaliadorEquipe(ABC):
    @abstractmethod
    def avaliar(self, colaboradores, criterio): ...

class IRepositorioEquipe(ABC):
    @abstractmethod
    def salvar(self, equipe: Equipe): ...

class INotificador(ABC):
    @abstractmethod
    def notificar(self, equipe, dest): ...

# ✔ O + L — Implementações intercambiáveis
class AvaliadorIA(IAvaliadorEquipe):
    def __init__(self, modelo: ModeloRL):
        self.modelo = modelo

    def avaliar(self, colaboradores, criterio):
        membros = self.modelo.predizer(colaboradores, criterio)
        return Equipe(membros=membros)

class AvaliadorRegras(IAvaliadorEquipe):
    # ✔ Fallback sem IA — mesmo contrato
    def avaliar(self, colaboradores, criterio):
        top = sorted(colaboradores,
                     key=lambda c: len(c.habilidades))
        return Equipe(membros=top[:criterio.tamanho])
```

```
# ✔ S — Cada classe tem 1 responsabilidade
class GeradorEquipes:
    # ✔ D — depende de abstrações (injeção)
    def __init__(self,
                 avaliador: IAvaliadorEquipe,
                 repositorio: IRepositorioEquipe
                 ):
        self.avaliador = avaliador
        self.repositorio = repositorio

    def gerar(self, colaboradores, criterio):
        equipe = self.avaliador.avaliar(colaboradores, criterio)
        self.repositorio.salvar(equipe)
        return equipe

# ✔ S — Relatório separado
class ServicoRelatorio:
    def __init__(self, repo: IRepositorioEquipe):
        self.repo = repo

    def gerar(self, equipe) → Relatorio:
        return Relatorio(dados=equipe.membros)

# ✔ S — Notificação separada
class ServicoNotificacao:
    def __init__(self, notif: INotificador):
        self.notif = notif

    def avisar(self, equipe, dest):
        self.notif.notificar(equipe, dest)

# ✔ Composição na aplicação
gerador = GeradorEquipes(
    avaliador=AvaliadorIA(ModeloRL()),
    repositorio=PostgresRepo(db_conn)
)
```

Melhorias aplicadas

- ✔ S — 1 classe, 1 responsabilidade
Gerador, Relatório e Notificação são classes independentes
- ✔ O — Extensível sem modificar
Novo avaliador: basta implementar IAvaliadorEquipe
- ✔ L — Avaliadores intercambiáveis
AvaliadorIA e AvaliadorRegras são substituíveis
- ✔ I — Interfaces mínimas
IAvaliador, IRepositorio e INotificador separados
- ✔ D — Injeção de dependência
GeradorEquipes recebe abstrações — 100% testável com mocks